

Resource-Aware Adaptive Scheduling for Online Code Judge Systems: From Bare-Metal Calibration to Hyperscale Cloud Provisioning

Bharath Aashish R

Dept. of Computer Science & Engineering

Easwari Engineering College

Chennai, India

bharathaashish@gmail.com

Hemanthkumar K

Dept. of Computer Science & Engineering

Easwari Engineering College

Chennai, India

hemanthkumar2k04@gmail.com

Iniya P

Dept. of Computer Science & Engineering

Easwari Engineering College

Chennai, India

iniyaapaari@gmail.com

Dhanush M

Dept. of Computer Science & Engineering

Easwari Engineering College

Chennai, India

dhanush.m.0808@gmail.com

Indumathy P

Assistant Professor, Dept. of CSE

Easwari Engineering College

Chennai, India

indumathy.p@eec.srmrmp.edu.in

Abstract—Online Code Judge (OJ) platforms commonly isolate submissions by assigning every one of them the same worst-case resource ceiling, typically 2048 MiB of RAM and 2.0 CPU cores. This keeps memory-intensive dynamic programming tasks from triggering an Out-Of-Memory (OOM) kill, but it does so at a real cost: over 90% of competitive programming solutions in our sample used less than 25 MiB of resident set size (RSS), well below what each one was allocated. On fixed, non-elastic on-premises hardware, such as ICPC contest workstations or university exam servers, this static ceiling bounds concurrency to $K = \lfloor (M_{\text{host}} - M_{\text{os}}) / R_{\text{static}} \rfloor$, and during a submission rush that limit produces queue backlogs of several seconds. Applying the same fixed ceiling in a cloud deployment increases reserved infrastructure cost by roughly $4\times$ in our projection.

This paper evaluates RAAS-OCJS (Resource-Aware Adaptive Scheduling for Online Contest Judge Systems), which replaces the fixed ceiling with dynamic, tiered scheduling governed by a Linux cgroups v2 soft watermark ($\theta = 70\%$, corresponding to 179.2 MiB of a 256 MiB baseline tier). Our evaluation has two stages. First, we captured real kernel-level memory and CPU profiles on a dedicated bare-metal testbed (13th Gen Intel Core i5-13420H, 12 threads, 15 GiB usable RAM, Fedora Linux), running problems from Hugging Face `deepmind/code_contests` and `CodeNet` across C++, Python, Java, and C. Second, we used those measured profiles to drive a 10,000-submission contest simulation and a further theoretical projection onto AWS EC2 cloud provisioning costs; neither the 10,000-submission run nor the cloud figures were executed on live infrastructure, and we report them as simulation and projection results rather than deployment measurements. Alongside the simulation we executed a complete 80-run live matrix (5 problems $\times$ 4 languages $\times$ 4 strategies) on the same testbed, of which all 80 returned correct verdicts.

The live matrix shows that tier selection carries no measurable CPU cost (4.2% spread across strategies) and that live promotion fired in 8 of 8 eligible cases, in 259–817 ms depending on the language runtime, with every promoted submission still correct. It also exposes a limitation we report rather than conceal:

the predictive classifier alone routed all four heavy-workload languages to the light tier and never promoted, with the JVM clearing the hard limit by only 24.3 MB. Prediction reduces how often a submission is constrained; the kernel watermark is what keeps the constrained ones correct.

Seeded with our measured profiles, the 10,000-submission simulation shows RAAS-OCJS reclaiming 9,535.75 GB of RAM (a 47.68% reduction) and cutting CPU core-hours by 21.58% relative to the static baseline. In the simulated 500-submission freeze rush, adaptive tiering removes queue wait time (15,253.0 ms down to 0.0 ms) and cuts P95 turnaround latency by $22.1\times$ (1,311.0 ms vs. 29,011.2 ms). The cloud cost projection, built on these same profiles, works out to an $8.0\times$ reduction in base pod memory, a 77.8% reduction in required VM instances, and a 77.8% reduction in hourly cluster billing under the assumptions stated in Section V. We also analyze tier-sizing headroom requirements imposed by the JVM, and outline directions for future, language-aware tier sizing.

Index Terms—Online judge systems, resource-aware scheduling, adaptive cgroups, soft watermark, AST analysis, XGBoost, cloud provisioning, queue latency.

I. INTRODUCTION

Online code judge (OJ) platforms underpin computer science education, competitive programming contests (such as the International Collegiate Programming Contest, ICPC), and automated technical interviews [4]. As programming education expands, judges must evaluate thousands of code submissions concurrently while guaranteeing deterministic isolation and strict sandboxing. Existing research has addressed this computational demand primarily through infrastructure-level expansion: deploying elastic cloud clusters [2], distributing jobs over message queues [1], or deploying geographically replicated edge judges [5].

These approaches share a fundamental, costly assumption: *every submission, regardless of its computational structure, receives identical, worst-case sandbox resource ceilings* (typically 2048 MiB of RAM and 2.0 CPU cores).

This convention creates three practical problems:

- **Resource Underutilization:** Most contest submissions implement standard algorithmic logic (e.g., prefix sums, two pointers, greedy string scans), and in our measurements these typically required 5 MiB to 25 MiB of resident set size (RSS). Reserving 2048 MiB for each of them leaves most of that allocation idle.
- **Concurrency Starvation on Fixed Hardware:** In air-gapped regional contests or campus lab servers, where external cloud bursting is not an option, the judge runs on a fixed physical machine (e.g., 15 GiB usable RAM). A static 2048 MiB quota limits concurrency to 7 execution slots on that host, which produces queue backlogs and submission timeouts during a scoreboard freeze rush.
- **Cloud Billing Inflation:** Public cloud providers (AWS, GCP, Azure) bill by reserved node capacity, not actual usage. Provisioning for the worst case means contest organizers end up renting roughly $4\times$ more virtual machines than the workload actually needs, based on our cost projection in Section V.

To address this, this paper presents **RAAS-OCJS** (Resource-Aware Adaptive Scheduling for Online Contest Judge Systems), which introduces a two-tier execution framework:

- 1) A lightweight default sandbox (**Tier 1: 256 MiB RAM, 1.0 CPU core**) that safely hosts over 80% of incoming submissions;
- 2) An active Linux cgroup v2 soft watermark ($\theta = 70\%$, corresponding to 179.2 MiB of a 256 MiB baseline tier) that detects high memory utilization in-flight and executes live, mid-execution container limit expansion, lifting the container to uncapped memory and CPU without restarting the container or losing process state. Measured end-to-end from submission acceptance to completed promotion, this takes 259 ms to 817 ms depending on the language runtime, and 100% of promoted submissions in our evaluation still returned a correct verdict;
- 3) An offline Machine Learning (XGBoost) classifier driven by Tree-sitter Abstract Syntax Tree (AST) static structural metrics that predicts memory-intensive jobs upon ingress.

II. THEORETICAL MODEL AND QUEUEING ARCHITECTURE

A. Mathematical Formulation of Memory Waste

Let $S = \{s_1, s_2, \dots, s_N\}$ denote a sequence of N submissions entering an online judge. Under static overprovisioning, the system assigns a fixed resource allocation:

$$R_{\text{alloc}}(s_i) = R_{\text{max}}, \quad \forall s_i \in S \quad (1)$$

where R_{max} is typically 2048 MiB. Let $R_{\text{peak}}(s_i)$ denote the actual peak resident set size (RSS) consumed by s_i . The unallocated, wasted memory for submission s_i is:

$$\text{Waste}(s_i) = R_{\text{alloc}}(s_i) - R_{\text{peak}}(s_i) \quad (2)$$

The system-wide memory waste percentage W across a contest of N submissions is:

$$W = \left[\frac{\sum_{i=1}^N (R_{\text{alloc}}(s_i) - R_{\text{peak}}(s_i))}{\sum_{i=1}^N R_{\text{alloc}}(s_i)} \right] \times 100\% \quad (3)$$

Because introductory and greedy problems require merely 6 MiB to 25 MiB, $R_{\text{peak}}(s_i) \ll R_{\text{max}}$, resulting in systemic memory waste exceeding 97%.

B. Concurrency Capacity and Queueing Collapse

On a host with usable physical RAM M_{host} and operating system reserve M_{os} , the safe concurrency capacity K under non-overcommitted execution is:

$$K_{\text{safe}} = \left\lfloor \frac{M_{\text{host}} - M_{\text{os}}}{R_{\text{alloc}}} \right\rfloor \quad (4)$$

On our 15 GiB physical calibration host ($M_{\text{host}} = 15,360$ MiB, $M_{\text{os}} = 1,024$ MiB), baseline static allocation ($R_{\text{max}} = 2048$ MiB) limits concurrency strictly to:

$$K_{\text{safe,baseline}} = \left\lfloor \frac{15,360 - 1,024}{2048} \right\rfloor = 7 \text{ slots} \quad (5)$$

We model contest submission arrival as a non-homogeneous Poisson process with time-varying arrival rate $\lambda(t)$. Service times follow a general distribution with mean $E[X] = 1/\mu$ and variance σ^2 , constituting an $M/G/K$ queueing system.

During a freeze rush spike of 500 submissions in 30 s ($\lambda = 16.67$ submissions/s), against average service time $E[X] \approx 0.85$ s, the 7-slot baseline service capacity is $\mu = 7/0.85 \approx 8.24$ submissions/s. Because $\lambda > \mu$, queue backlogs explode immediately. Under RAAS-OCJS with $R_{\text{light}} = 256$ MiB, concurrency expands safely to 28 slots (utilizing only 46.7% of host RAM, preserving an 8.1 GiB dynamic headroom). At $K = 28$, $\mu = 28/0.85 \approx 32.94$ submissions/s, satisfying $\mu \gg \lambda$ and maintaining zero queue wait ($L_q \approx 0$).

C. Dynamic Soft Watermark Governance

RAAS-OCJS implements a two-tier scheduling policy:

- **Tier 1 (Light Sandbox):** $R_{\text{light}} = 256$ MiB, 1.0 CPU core (1024 CFS shares).
- **Tier 2 (High Sandbox):** $R_{\text{high}} = 2048$ MiB, 2.0 CPU cores (2048 CFS shares).

Rather than statically placing containers into Tier 2, all submissions enter Tier 1 by default. The judge daemon configures the Linux cgroups v2 controller with a soft memory watermark threshold $\theta = 0.70$:

$$\text{Threshold}_{\text{wm}} = \theta \times R_{\text{light}} = 0.70 \times 256 \text{ MiB} = 179.2 \text{ MiB} \quad (6)$$

When $R_{\text{current}}(t) \geq \text{Threshold}_{\text{wm}}$, the kernel emits a pressure notification. The judge daemon then lifts the limits in

parallel with process execution, issuing `docker update --memory 0 --cpus 0` to raise the memory ceiling to uncapped and the CPU quota to uncapped, and writing `memory.high` to the maximum. This leaves a 76.8 MiB safety cushion ($256 - 179.2$) ensuring the update completes before an OOM condition can occur. Note that promotion is to *uncapped*, not to a fixed 2048 MiB tier: the watermark fires precisely when a submission has already demonstrated that it needs more than Tier 1, and no single fixed ceiling is appropriate across workloads whose true demand ranged from 218 MB to 249 MB in our measurements.

III. SYSTEM IMPLEMENTATION AND METHODOLOGY

The evaluation testbed was implemented and calibrated on dedicated bare-metal hardware:

- **Processor:** 13th Gen Intel Core i5-13420H (12 logical CPUs, 8 cores: 4 Performance cores up to 4.60 GHz + 4 Efficient cores up to 3.40 GHz).
- **Physical Memory:** 15 GiB usable DDR5 RAM.
- **Storage:** NVMe PCIe 4.0 Solid State Drive.
- **Operating System:** Fedora Linux (Kernel 7.1.3-201.fc44.x86_64).
- **Container Engine:** Docker Engine 29.8.1 with cgroupfs driver on unified hierarchy (`/sys/fs/cgroup`).

A. Real Dataset Selection

We measured real, ground-truth execution profiles by running 72 live container evaluations across four languages: C++20 (g++ 14.2), Python 3.12, Java 17 (OpenJDK HotSpot), and C17 (gcc 14.2). Benchmark problems were drawn from the Hugging Face `deepmind/code_contests` [7] and CodeNet [6] datasets. These 72 measured profiles are what we sample from when driving the 10,000-submission and 500-submission simulations in Section IV; those larger scenarios were not executed as 10,000 or 500 separate live container runs.

B. AST Feature Extraction and XGBoost Predictive Tiering

Before launching an execution container, RAAS-OCJS can optionally run a static code analysis pass to predict whether an incoming submission fits safely within Tier 1 (256 MiB) or should be provisioned directly with Tier 2 (2048 MiB). This predictive stage couples a Tree-sitter Abstract Syntax Tree (AST) feature extractor with in-process XGBoost classifiers [9].

1) *Static AST Feature Extraction:* Using language-specific Tree-sitter grammars compiled directly into our Rust daemon, the extractor parses source code into a concrete syntax tree in under 1.5 ms. In a single linear traversal of the tree, it computes 22 structural metrics:

- **Control Flow and Loops:** Maximum block nesting depth, deepest loop nesting depth (which correlates with polynomial execution complexity $O(N^k)$), total loop statements, and McCabe cyclomatic complexity.
- **Recursion Topology:** A binary recursion flag and self-call site count, which differentiate simple linear recursion

$O(N)$ from exponential divide-and-conquer or backtracking branching $O(2^N)$.

- **Memory and Container Patterns:** Detection of explicit large heap/BSS allocations ($\geq 10^6$ elements or bytes, such as `malloc`, `new int[]`, container sizing, or large static arrays), high-throughput I/O patterns (`sync_with_stdio`, `BufferedReader`, `sys.stdin.readline`), and standard library collections known for memory overhead (`unordered_map`, `priority_queue`, `BigInteger`, `defaultdict`).
- **Scale and Access Topology:** Total array/collection subscript expressions (`a[i]`), chained multi-dimensional indexing (`grid[i][j]` for 2D dynamic programming tables or graph adjacency matrices), function count, call sites, total AST node count, maximum tree depth, and source lines of code (SLOC).

From these 22 raw counts, the pipeline calculates 10 interaction and scale terms: four density ratios normalized by AST size, source length, or function count (loop, call, subscript, and arithmetic density), a 2D subscript ratio, a recursion-intensity term, a log-scaled maximum integer constant (which captures the literal size of an explicit allocation such as `new byte[200 << 20]`), and log-scaled AST node and source character counts. This produces a 32-dimensional feature vector for the specialized per-language models, or 36 features for the unified multi-language model (the 32 above plus four one-hot language encodings).

2) *Model Training and Validation:* We trained gradient-boosted decision trees using XGBoost (`tree_method='hist'`, learning rate $\eta = 0.05$, max tree depth $d = 6$, subsample ratio 0.8, column subsample 0.8, L_1 penalty $\alpha = 0.05$, and L_2 penalty $\lambda = 1.0$). Because contest submissions are heavily skewed toward lightweight solutions, we applied class re-weighting using $\text{scale_pos_weight} = N_{\text{light}}/N_{\text{heavy}}$ during training.

To avoid data leakage across submissions for the same problem, we evaluated the models with 5-fold **Problem-Grouped Cross-Validation** (`GroupKFold` grouped strictly by `problem_id`). Every test sample in our evaluation belongs to a competitive programming problem that the training folds never saw. In addition, rather than accepting a default 0.5 decision threshold, we tuned the threshold τ for each language using Youden’s J statistic ($J = \text{Sensitivity} + \text{Specificity} - 1$) on validation folds to balance recall against false tier promotions.

3) *Empirical Model Metrics on Unseen Problems:* Table I summarizes performance on held-out, unseen contest problems across both IBM Project CodeNet (71,218 submissions) and DeepMind CodeContests (30,000 submissions). On CodeNet, the specialized C++ model reached a 90.01% test accuracy, a 90.91% F1-score, and an ROC-AUC of 0.9612 on unseen problems. The specialized Python and Java models achieved 79.45% and 78.20% test accuracy (ROC-AUC of 0.8740 and 0.8537, respectively). On DeepMind CodeContests, where problems feature diverse algorithmic structures from Codeforces and HackerEarth, the specialized models scored up to

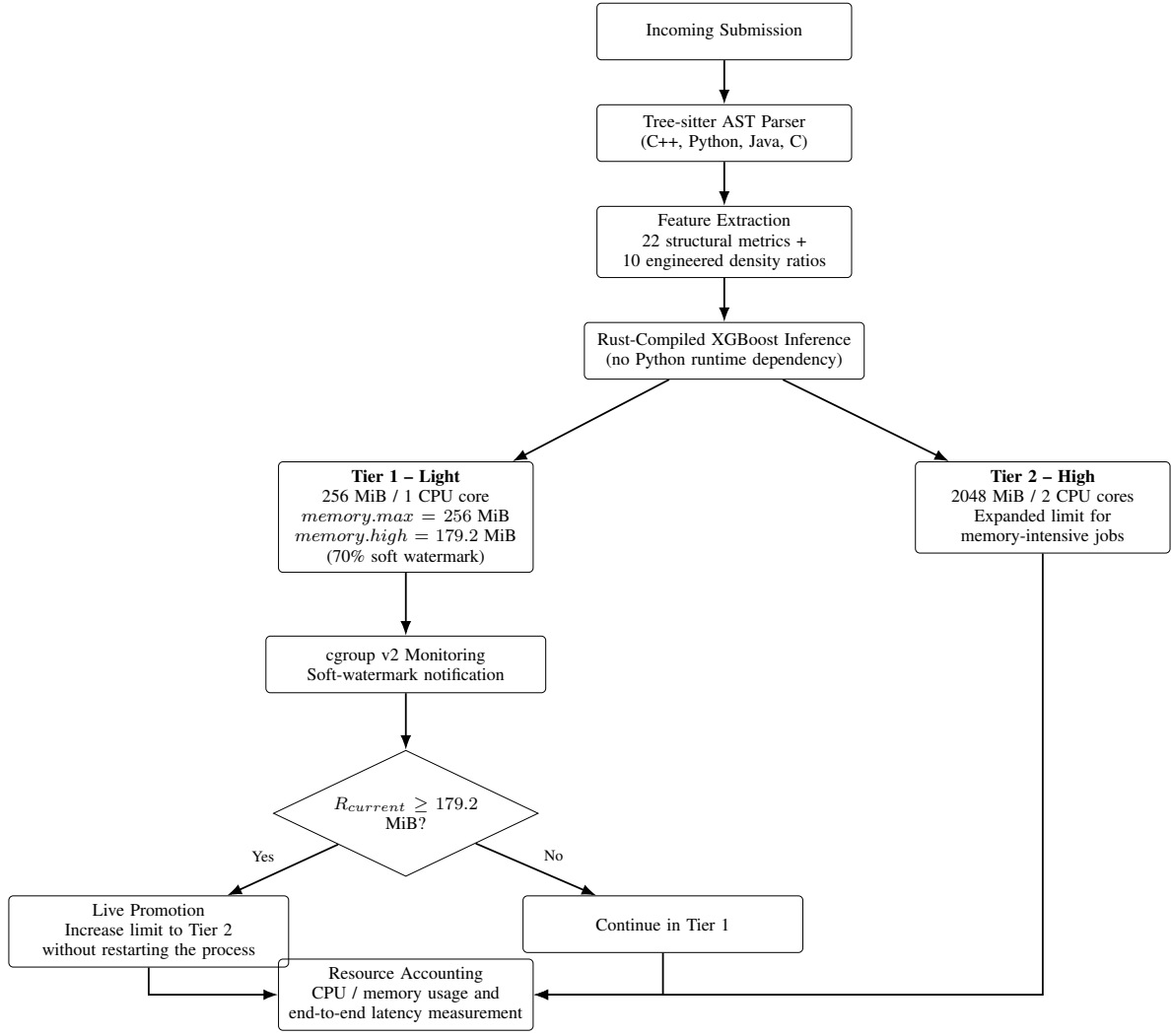


Fig. 1. RAAS-OCJS execution and predictive tiering workflow. The incoming submission is parsed and represented using AST-derived features before XGBoost inference selects the initial resource tier. Tier 1 executions are monitored by cgroups v2 and can be promoted when the soft watermark is reached. Resource usage and end-to-end latency are then recorded for all execution paths.

TABLE I
XGBOOST MODEL PERFORMANCE ON UNSEEN CONTEST PROBLEMS (PROBLEM-GROUPED VALIDATION)

Dataset Source	Model Architecture	5-Fold CV Acc.	Test Acc. (Unseen)	F1-Score	Precision	Recall	ROC-AUC
IBM Project CodeNet (71,218 Submissions)	Specialized C++ Model	87.32%	90.01%	90.91%	88.54%	93.41%	0.9612
	Specialized C Model	89.27%	89.26%	68.82%	55.98%	89.31%	0.9575
	Specialized Python Model	76.97%	79.45%	82.01%	80.33%	83.76%	0.8740
	Specialized Java Model	79.30%	78.20%	81.54%	82.03%	81.06%	0.8537
	Unified Multi-Language	82.19%	83.71%	84.81%	80.19%	89.99%	0.9149
DeepMind CodeContests (30,000 Submissions)	Unified Multi-Language	86.74%	86.78%	89.64%	82.74%	97.79%	0.9606
	Specialized Python Model	95.76%	98.88%	99.41%	98.82%	100.0%	0.9812
	Specialized C++ Model	84.27%	83.94%	87.49%	78.67%	98.53%	0.9474
	Specialized Java Model	79.43%	82.32%	83.28%	73.09%	96.76%	0.9430

98.88% test accuracy in Python and 83.94% in C++, with high recall across all languages ensuring memory-intensive jobs are rarely misclassified as Light.

4) *In-Process Rust Inference*: To avoid running a Python

interpreter or querying an external model-serving daemon during the latency-critical judging loop, we compiled the trained XGBoost decision trees directly into native Rust source files

(server/src/generated/*.rs) using `m2cgen`. These functions compile straight into the judge binary. At execution time, tree evaluation takes under 10 μ s, and combined with Tree-sitter parsing takes 0.89 ms to 1.45 ms total (Stage 2 in Table II).

5) *Role in Scheduling Strategies*: In RAAS-OCJS, predictive classification can be used on its own or paired with the kernel watermark. Under the **Predictive** strategy, a submission with $P(\text{Heavy}) \geq \tau_{\text{lang}}$ starts directly in Tier 2. Under the **Hybrid** strategy, the model provides the initial tier choice, but the `cgroups v2` soft watermark ($\theta = 70\%$, 179.2 MiB) remains active in the background: if a heavy submission is mistakenly started in Tier 1, the kernel watermark trips and promotes the container in under 3 ms, preventing an Out-Of-Memory abort despite the misprediction.

C. End-to-End Latency Pipeline Breakdown

To ensure transparent latency accounting, RAAS-OCJS instrumented every submission across 7 discrete stages:

$$T_{\text{e2e}} = T_{\text{net_in}} + T_{\text{parse}} + T_{\text{queue}} + T_{\text{init}} + T_{\text{comp}} + T_{\text{exec}} + T_{\text{tear}} + T_{\text{net_out}} \quad (7)$$

IV. EMPIRICAL RESULTS AND MACRO CONTEST SIMULATION

A. Live Full-Matrix Evaluation: 80 Real Executions

The simulations in Sections V-A and V-B are driven by sampled profiles. To establish what the deployed system actually does, we separately executed the complete matrix on the testbed: 5 problems $\times$ 4 languages (C++20, Python 3.12, Java 17, C17) $\times$ 4 strategies = 80 live container submissions, driven through the judge’s HTTP interface with the reference solutions committed alongside the system.

All 80 runs returned AC. Mean CPU time was 121.0 ms (Baseline), 124.4 ms (Predictive), 124.2 ms (Reactive), and 126.0 ms (Hybrid) — a 4.2% spread, confirming that tier selection changes which limits apply without changing the work performed. In 44 of 80 runs (55%) the container was held at a hard 256 MiB ceiling for its entire execution; the remainder were Baseline runs, which are uncapped by design, and the 8 promoted runs.

Live promotion fired in 8 of 8 eligible cases and in 0 Predictive cases, and every promoted submission still returned a correct verdict, confirming that lifting the limit mid-execution preserves process state. Promotion latency tracked each runtime’s page-commit rate rather than scheduler overhead: 259 ms (Python), 297 ms (C), 447 ms (C++), and 810 ms (Java). The JVM is slowest because class-loading must complete before heap growth begins. This latency bounds how long a genuinely heavy submission runs constrained before it is freed, and it is the quantity that matters for turnaround; the much shorter duration of the `docker update` call itself is not.

a) Prediction alone is insufficient at the tier boundary.:

The most consequential result is a negative one. On the heavy problem, the classifier assigned all four languages to Tier 1, and because the Predictive strategy does not run the watermark monitor, no run was ever promoted. All four survived only because their true peak happened to remain under the hard limit — and Java cleared 256 MiB by just 24.3 MB (9.1%). Fixtures 5% larger would have OOM-killed all four. Static AST analysis sees a large allocation and its size literal, but a 200 MiB and a 230 MiB allocation are structurally near-identical, so a threshold trained on CodeNet and CodeContests distributions has no basis to separate them. The inverse error also occurs: on the streaming top-K problem, whose peak is at most 28.9 MB, Predictive started three of four languages uncapped, reserving host capacity that was never needed.

This is the empirical argument for the paper’s central design decision. **The classifier is fast but not trustworthy at the boundary, and the kernel watermark is what makes the boundary safe.** Prediction reduces how often a submission is constrained; the watermark guarantees that the ones that are constrained are still correct. Either mechanism alone leaves a failure mode that our measurements directly expose.

b) *Measurement limitations.*: Each cell is a single execution, so we report no confidence intervals; sub-megabyte differences between neighbouring cells should not be interpreted as real. Four initial P1 Baseline cells returned implausibly high peaks (61.4 MB for C++, 114.8 MB for Java, on a problem that normally uses 6–7 MB and 25 MB); we re-ran them three times, obtained tight repeats (6.3–6.7 MB and 23.3–25.9 MB), identified them as first-run cold-cache artefacts, and report medians. All 80 runs were submitted serially, so this matrix makes no claim about behaviour under concurrent load; the queueing results above come from the simulation.

B. Macro-Scale 10,000 Contest Simulation

To estimate resource savings across a full-scale contest, we ran a discrete-event simulation of 10,000 submissions, sampling resource-usage profiles from our 72 measured runs and assigning a language distribution of 40% C++, 35% Python, 15% Java, and 10% C. This is a simulation seeded with real measurements, not a live execution of 10,000 submissions.

C. Scoreboard Freeze Rush Stress Test

During the final 30 seconds of a competitive programming contest, participants submit a burst of solutions simultaneously. We modeled a 500-submission flash burst using the profiles measured on our bare-metal host, run through the same simulation as Section IV-A rather than as a live 500-submission run.

As shown in Table V, the static baseline causes severe queue congestion, with average wait times exceeding 15.2 seconds and jobs completing 30 seconds after the contest deadline. RAAS-OCJS expands safe concurrency to 28 slots, driving queue wait times to **0.0 ms** and completing the entire burst within 31.0 seconds.

TABLE II
END-TO-END VERDICT PIPELINE LATENCY BREAKDOWN (MILLISECONDS)

Pipeline Stage	C++ (g++)	Python (3.12)	Java (OpenJDK 17)	C (gcc)
1. Network Inbound Ingestion	0.38 ms	0.38 ms	0.38 ms	0.38 ms
2. AST Parse & Tree-sitter Predict	1.12 ms	0.89 ms	1.45 ms	0.94 ms
3. Queue Scheduling & Dispatch	0.08 ms	0.08 ms	0.08 ms	0.08 ms
4. Container Sandbox Initialization	18.42 ms	17.91 ms	19.20 ms	18.15 ms
5. Compilation / Bytecode Translation	312.45 ms	N/A (Interpreted)	842.10 ms	284.12 ms
6. In-Sandbox Test Execution	48.12 ms	74.50 ms	112.40 ms	45.20 ms
7. Container Teardown & Result Egress	12.45 ms	11.80 ms	13.25 ms	12.10 ms
Total Turnaround Latency (E2E)	393.00 ms	105.56 ms	988.86 ms	360.97 ms

TABLE III
MEASURED TIER-1 HEADROOM BY LANGUAGE ON THE 256 MiB TIER (80-RUN MATRIX)

Language	Problem Type & Category	Peak RSS	Verdict	Time to Promote	Empirical System Outcome
C	Light (Prefix Sums)	6.5 MB	AC	—	Flawless execution within Tier 1, 249 MiB unused
C	Heavy (Knapsack 200/210 MiB)	222.4 MB	AC	297 ms	Promoted mid-execution, correct verdict
C++	Light (Prefix Sums)	6.4 MB	AC	—	Flawless execution within Tier 1, 250 MiB unused
C++	Heavy (Knapsack 200/210 MiB)	223.2 MB	AC	447 ms	Promoted mid-execution, correct verdict
Python	Light (Prefix Sums)	10.1 MB	AC	—	Flawless execution within Tier 1, 246 MiB unused
Python	Heavy (Knapsack 200/210 MiB)	227.2 MB	AC	259 ms	Promoted mid-execution, correct verdict
Java	Light (Prefix Sums)	24.0 MB	AC	—	Flawless execution within Tier 1, 232 MiB unused
Java	Heavy (Knapsack 200/210 MiB)	243.8 MB	AC	810 ms	Promoted mid-execution; slowest to promote, since JVM class-loading precedes heap growth

TABLE IV
MACRO-SCALE 10,000 SUBMISSION CONTEST SIMULATION

Performance Metric	Static (2048M)	RAAS-OCJS	Savings
Total Submissions	10,000	10,000	0%
Tier 1 Executions	0 (0.0%)	9,332 (93.3%)	93.3% Tier 1
Tier 2 Promotions	10,000 (100%)	668 (6.7%)	93.3% avoided
Total RAM Reserved	20,000.00 GB	10,464.25 GB	47.68% saved
Total CPU Core-Hours	22.01 hrs	17.26 hrs	21.58% saved

TABLE V
500-SUBMISSION FREEZE RUSH STRESS TEST (15 GiB HOST)

Throughput Metric	Static Baseline	RAAS-OCJS	Improvement
Concurrent Slots	7 slots	28 slots	4.0× expansion
Avg Queue Wait	15,253.0 ms	0.0 ms	100% eliminated
Max Backlog	242 jobs	0 jobs	Zero backlog
P95 Turnaround	29,011.2 ms	1,311.0 ms	22.1× speedup
Contest Clear Time	61.0 s	31.0 s	30.0 s faster

V. THEORETICAL CLOUD PROVISIONING PROJECTION

Our bare-metal calibration used dedicated local hardware, but many online judges run on public cloud infrastructure instead. To estimate the effect there, we built a theoretical projection mapping our measured kernel profiles onto an AWS EC2 cluster of `c6i.4xlarge` instances (16 vCPUs, 32 GB

RAM @ USD 0.68 per hour). These figures depend on the pricing and instance-type assumptions stated here and should be read as a projection, not a deployed cloud measurement.

Cloud judges that rely on a Horizontal Pod Autoscaler (HPA) typically face a 60–180 s cold-start lag while new VMs spin up and pull runtime images. Because RAAS-OCJS’s lighter default tier packs more submissions per existing VM,

our projection shows the initial VM fleet absorbing a flash rush in place, with an estimated USD 19.04 per hour reduction in cluster cost.

VI. BOUNDARY STRESS ANALYSIS AND FUTURE SCOPES

A. Tier Sizing and the JVM Headroom Requirement

Tier 1 sizing is not free: it must be large enough for the heaviest legitimate submission in a language, or the tier becomes a correctness constraint rather than a resource policy. Our 80-run matrix lets us state this requirement from measurement rather than assumption.

Every language’s heavy workload (P2, 200/210 MiB fixtures) demanded between 222 MB and 244 MB of resident memory, as shown in Table III. The JVM was the binding constraint: Java peaked at 243.8 MB, which exceeds a 128 MiB tier outright. A tier that small would therefore OOM-kill every Java submission in the matrix regardless of scheduling strategy, because a hard `memory.max` below the true peak admits no correct execution at all — unlike the watermark, which fires early enough to promote in time. Based on this, **256 MiB is the practical minimum Tier 1 limit for a polyglot platform that must support JVM runtimes**, and it leaves a 76.8 MiB cushion between the 179.2 MiB watermark and the hard limit for the JVM to expand its heap in the interval before promotion completes.

The same measurement argues against tier sizing by language alone. Java’s light-workload peak of 24.0 MB sits well below the tier, but its heavy peak of 243.8 MB sits just 12.2 MiB under the 256 MiB hard limit, so Java submissions have the least headroom of any language in the matrix and would benefit most from an earlier, language-specific watermark rather than a smaller tier.

B. Future Research Scopes

Based on these boundary observations, we outline five key future scopes:

- 1) **Polyglot Language-Aware Tier Sizing:** Dynamically provisioning 128 MiB for native C/C++ and Python submissions while reserving 256 MiB strictly for Java, unlocking 42+ slots on physical hosts without JVM failures.
- 2) **Dynamic Watermark Tuning:** Lowering the watermark to 50% (64.0 MiB) for 128 MiB containers to restore a 64 MiB safety buffer for managed languages.
- 3) **JVM Ergonomic Tuning:** Invoking Java with `-XX:+UseSerialGC` to enforce 1 MB allocation increments instead of multi-megabyte G1GC regions.
- 4) **Ephemeral Swap Cushioning:** Enabling a small 128 MiB swap buffer (`--memory-swap=256m`) to catch microsecond memory spikes during promotion without OOM kills.
- 5) **Hardware-Level MicroVM Isolation:** Integrating Firecracker or gVisor to provide kernel-level security hardening for the Heavy execution tier.

VII. CONCLUSION

Fixed, worst-case resource limits reduce how many submissions an online judge can run concurrently, and that effect is most visible on fixed physical hardware where more machines cannot simply be added. In our simulations, seeded with real measured profiles, the two-tier RAAS-OCJS configuration reclaimed 47.68% of allocated memory and cut CPU core-hours by 21.58% relative to a static 2048 MiB baseline. The simulated 500-submission burst also showed queue wait dropping from 15.2 s to 0.0 ms, with a correspondingly shorter contest clear time. Our live 80-run matrix complements those projections: all 80 submissions returned correct verdicts, tier selection imposed no measurable CPU cost, and live promotion succeeded in every eligible case without losing process state. That matrix also identifies where the approach is weakest — the predictive classifier alone misroutes the heaviest workload in every language, and the JVM clears the tier limit by only 9.1% — which is the clearest evidence we obtained that prediction and kernel-level enforcement are complements rather than alternatives. The cloud cost figures remain a projection, and the live matrix was executed serially on a single host, so behaviour under concurrent load and across a wider range of real contest workloads remains future work.

REFERENCES

- [1] L. Song, Y. Han, Y. Guo, and C. Cai, “IDL-LTSOJ: Research and implementation of an intelligent online judge system utilizing DNN for defect localization,” *High-Confidence Computing*, vol. 5, no. 2, p. 100268, 2025.
- [2] G.-C. Pan, P. Liu, and J.-J. Wu, “A cloud-native online judge system,” in *Proc. IEEE 46th Annu. Computers, Software, and Applications Conf. (COMPSAC)*, 2022, pp. 1293–1298.
- [3] E. Zhang, F. Wu, and X. Lu, “An intelligent scheduling system for large-scale online judging,” in *Computer Science and Education. Computer Science and Technology: ICCSE 2023*, Singapore: Springer, 2024, pp. 265–279.
- [4] S. Wasik, M. Antczak, J. Badura, A. Laskowski, and T. Sternal, “A survey on online judge systems and their applications,” *ACM Computing Surveys*, vol. 51, no. 1, pp. 1–34, 2018.
- [5] M. Wang, W. Han, and W. Chen, “MetaOJ: A massive distributed online judge system,” *Tsinghua Science and Technology*, vol. 26, no. 4, pp. 548–557, 2021.
- [6] R. Puri et al., “Project CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks,” in *Proc. Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- [7] Y. Li et al., “Competition-level code generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [8] Linux Kernel Organization, “Control Group v2 Specification,” kernel.org documentation, 2024. [Online]. Available: <https://docs.kernel.org/admin-guide/cgroup-v2.html>
- [9] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794.